PDM MEMS MICROPHONE INTERFACING WITH RASPBERRY PI AND PDM-TO-PCM AUDIO CONVERSION



NATIONAL INSTITUTE OF TECHNOLOGY ROURKELA

DEPARTMENT OF ELECTRICAL ENGINEERING

Research Project I

EE 4999

Submitted by

P Sridevi – 123EE0846

Aakhyarika Priyadarsani – 123EE0315

Under the Guidance of

Prof. Rakesh Sinha

Academic Year 2026–27

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide Prof. Rakesh Sinha and the Department of Electrical Engineering, National Institute of Technology Rourkela, for their guidance and support throughout this project. We are also thankful to our institute for providing the necessary facilities and resources required to carry out the experimental work.

ABSTRACT

This project focuses on the interfacing and acquisition of a digital PDM MEMS microphone using a Raspberry Pi 4B. The work investigates the acquisition of the high-frequency one-bit PDM output and its conversion into usable PCM audio. Two approaches are being explored. The first approach uses the Raspberry Pi PCM peripheral along with the `pdm_start` and `pdm_rec` utilities for microphone clock generation and data acquisition. The second approach uses the SPI0 peripheral to directly acquire the PDM bitstream, followed by software-based PDM-to-PCM conversion using CIC decimation, FIR low-pass filtering, and DC blocking.

Initial experimental results from the PCM-peripheral-based approach demonstrate successful microphone operation and audio tone capture. The SPI-based approach has also been developed with a digital decimation chain for obtaining PCM audio. The two approaches will be further evaluated and compared in terms of acquisition, processing, audio quality, and practical limitations.

Contents

1 INTRODUCTION 4

2 PROBLEM STATEMENT 4

3 BACKGROUND 4
- 3.1 Raspberry Pi 4B 4
- 3.2 PDM MEMS Microphone 5
- 3.3 Project Objectives 5

4 SYSTEM OVERVIEW 5

5 METHOD 1: SPI-BASED PDM ACQUISITION AND CONVERSION 6
- 5.1 PDM Signal and Conversion Principle 6
- 5.2 Raspberry Pi SPI Acquisition 6
- 5.3 PDM-to-PCM Signal Processing 6
  - 5.3.1 CIC Decimation 7
  - 5.3.2 FIR Low-Pass Filtering 7
  - 5.3.3 DC Blocking and PCM Output 7
- 5.4 Conversion Parameters and Validation 7
- 5.5 Experimental Results 8

6 METHOD 2: PDM ACQUISITION USING PCM PERIPHERAL 8
- 6.1 Clock Generation and Data Acquisition 8
- 6.2 Signal Analysis 9
- 6.3 Experimental Results 9

1 INTRODUCTION

Digital MEMS microphones are widely used in embedded audio systems due to their compact size, low power consumption, and digital output. A PDM microphone represents audio using a high-frequency one-bit stream whose bit density varies with sound pressure.

This project focuses on recording and processing audio using a PDM MEMS microphone and a Raspberry Pi 4B. The Raspberry Pi generates the required clock, acquires the PDM output, and processes it to obtain usable audio that can be stored and played back as a conventional audio file. Since the PDM clock frequency is considerably higher than the desired audio sampling frequency, the acquired signal must be processed through appropriate conversion and sampling techniques before it can be used as conventional digital audio. In this report two methods are being investigated for PDM acquisition and conversion. The first uses the Raspberry Pi's PCM peripheral, while the second uses the SPI0 interface for direct PDM acquisition followed by software-based PDM-to-PCM conversion.

2 PROBLEM STATEMENT

The PDM output of a digital MEMS microphone is a high-frequency, one-bit stream that cannot be directly used as conventional digital audio.The objective of this project is to develop and evaluate a Raspberry Pi 4B-based system capable of acquiring the PDM output of a MEMS microphone and obtaining usable PCM audio.

3 BACKGROUND

PDM encodes audio as the density of "1" bits in a high-frequency binary stream, with the audio information concentrated at lower frequencies and quantization noise shifted to higher frequencies. Since the PDM clock frequency is much higher than the desired audio sampling frequency, the acquired signal requires low-pass filtering and decimation to reduce its sampling rate and remove unwanted high-frequency components. The project therefore investigates both PDM signal acquisition and the subsequent conversion or analysis required to obtain usable digital audio.

3.1 Raspberry Pi 4B

The Raspberry Pi 4 Model B is a compact single-board computer with GPIO and interfaces such as SPI, I2C, UART, and PCM, making it suitable for data acquisition, signal processing, and hardware interfacing. In this project, it serves as the main processing and data acquisition platform for the PDM MEMS microphone, generating the required clock, receiving PDM data, and processing it into usable audio.

3.2 PDM MEMS Microphone

The microphone used is the SPH0641LU4H-1 digital MEMS microphone, which integrates an acoustic sensor, signal conditioning circuitry, and a sigma-delta modulator to generate a one-bit PDM output. It has five pins: 3.3V, GND, CLK, DATA, and SELECT. The microphone requires an external clock through CLK and provides the PDM signal through DATA, while SELECT determines the clock edge used for data transmission. The PDM output is a high-frequency sequence of logic "1" and "0" bits, where the bit density represents the audio signal. The microphone operates from a low-voltage supply and is directly interfaced with the Raspberry Pi 4B through the 3.3V, ground, clock, data, and SELECT connections. The physical interface and signal acquisition process are described in later sections

3.3 Project Objectives

The main objectives of this project are:

- To interface a PDM MEMS microphone with the Raspberry Pi 4B.
- To acquire the PDM signal from the microphone.
- To convert the acquired PDM signal into PCM audio.
- To record and play back the obtained audio signal.

4 SYSTEM OVERVIEW

The system consists of a Raspberry Pi 4B interfaced with a PDM MEMS microphone. The Raspberry Pi is used to provide the required clock, acquire the PDM data, and process it to obtain PCM audio. The system is implemented using Raspberry Pi OS Bookworm (Debian 12).

Table 1: PDM Microphone to Raspberry Pi 4B Connections

Microphone Pin	**Raspberry Pi Pin**	**Function**
3.3V	Pin 1	Power
GND	Pin 6	Ground
CLK	GPIO 18 (Pin 12)	Clock
DATA	GPIO 20 (Pin 38)	PDM data
SELECT	GND (Pin 9)	Selection

Two methods are being investigated for PDM acquisition and conversion. The first uses the Raspberry Pi's PCM peripheral, while the second uses the SPI0 interface for direct PDM acquisition followed by software-based PDM-to-PCM conversion.

5 METHOD 1: SPI-BASED PDM ACQUISITION AND CONVERSION

5.1 PDM Signal and Conversion Principle

The Knowles SPH0641LU4H-1 is a digital MEMS microphone that produces a one-bit PDM output. It integrates an acoustic sensor, low-noise buffer, and sigma-delta modulator. Its DATA pin carries one bit per clock cycle, with increasing sound pressure represented by a higher density of ones. The PDM stream is oversampled at a few-megahertz clock, with audio information concentrated at low frequencies and quantisation noise shifted towards higher frequencies. Therefore, the PDM signal requires low-pass filtering and decimation before it can be used as ordinary audio.

PDM-to-PCM conversion consists of low-pass filtering followed by sample-rate reduction. In the implemented chain, the PDM bits are unpacked and mapped from $\{0, 1\}$ to $\{-1, +1\}$, followed by a fifth-order CIC decimator. A 95-tap FIR low-pass filter removes remaining out-of-band components before final decimation, and a DC-blocking filter produces 16-bit mono PCM audio. For a PDM clock of 2.4038 MHz and 20 kHz audio bandwidth, the oversampling ratio is approximately 60.1.

5.2 Raspberry Pi SPI Acquisition

The PDM stream is acquired using the Raspberry Pi 4B SPI0 peripheral. Since user-space Python cannot reliably sample GPIO at megahertz rates due to operating-system scheduling, SPI provides hardware-based clock generation and sampling. The microphone is powered from 3.3 V, with its clock connected to the SPI clock and DATA connected to SPI MISO. With SELECT grounded, the interface is compatible with SPI mode 0, and each received byte contains eight consecutive PDM samples.

With a requested clock of approximately 2.4 MHz, an SPI divider of 208 gives an actual PDM clock of 2.4038 MHz:

$$f_{clk} = \frac{500\,\text{MHz}}{208} = 2.4038\,\text{MHz} \tag{1}$$

The actual clock is used for calculating the PCM output sampling rate. Data are captured in blocks of 65,536 bytes, corresponding to 524,288 PDM bits. Two initial blocks are discarded while the microphone settles. Short gaps may occur between SPI block reads, which is an acquisition limitation.

5.3 PDM-to-PCM Signal Processing

The received SPI bytes are unpacked into individual PDM bits in time order and converted from $\{0, 1\}$ to $\{-1, +1\}$. The resulting bipolar data are passed through the digital decimation chain.

5.3.1 CIC Decimation

A fifth-order CIC filter with decimation factor $R = 25$ is used as the first stage. It reduces the sampling rate from 2.4038 MHz to approximately 96.15 kHz without requiring conventional multipliers. The subsequent FIR stage handles the associated passband droop.

The CIC transfer function is:

$$H(z) = \left[\frac{1 - z^{-R}}{1 - z^{-1}}\right]^N \tag{2}$$

where $N = 5$ is the filter order and $R = 25$ is the decimation factor.

5.3.2 FIR Low-Pass Filtering

After CIC decimation, a 95-tap Hamming-windowed sinc FIR low-pass filter with an 18 kHz cutoff is applied. It removes remaining out-of-band components before the final factor-of-two decimation. The response is within 0.2 dB up to 5 kHz, with approximately 0.8 dB droop at 10 kHz and 1.7 dB at 15 kHz, while attenuation exceeds 50 dB above 20 kHz.

5.3.3 DC Blocking and PCM Output

The FIR output is passed through a one-pole DC-blocking filter:

$$y[n] = x[n] - x[n-1] + 0.995y[n-1] \tag{3}$$

The pole value of 0.995 gives a corner frequency of approximately 38 Hz. The signal is then scaled to approximately 80% of full scale and written as 16-bit mono PCM WAV audio. The final sampling rate is:

$$f_{out} = \frac{f_{clk}}{25 \times 2} = 48.0769\,\text{kHz} \tag{4}$$

5.4 Conversion Parameters and Validation

Table 2: PDM-to-PCM Conversion Parameters

Parameter	**Value**
SPI/PDM clock	2.4038 MHz
CIC decimator	Order 5, factor 25
Intermediate rate	96.15 kHz
FIR low-pass	95 taps, 18 kHz cutoff
Final decimation	Factor 2
PCM sampling rate	48.0769 kHz
DC blocker	0.995 / $\sim$38 Hz
Output format	16-bit mono WAV

The effect of CIC order was also evaluated. A first-order CIC gave an in-band SNR of 25.9 dB, while third- and fifth-order configurations achieved 68.9 dB and 70.0 dB, respectively. The fifth-order CIC was therefore selected for the conversion chain.

5.5 Experimental Results

A complete PDM-to-PCM conversion chain was implemented using the Raspberry Pi SPI0 peripheral and Python/NumPy-based signal processing. The chain consists of bit unpacking, fifth-order CIC decimation by 25, 95-tap FIR filtering, factor-of-two decimation, and DC blocking. The resulting output is 16-bit mono PCM at approximately 48.077 kHz. Simulation recovered a 1 kHz tone at 1000.0 Hz with an amplitude of 0.301 and achieved an in-band SNR of 70.0 dB.

The main practical limitation is the short gap between SPI blocks during acquisition, while the digital processing chain successfully performs the required conversion from the high-rate PDM stream to usable PCM audio.

6 METHOD 2: PDM ACQUISITION USING PCM PERIPHERAL

The first approach uses the Raspberry Pi's built-in PCM peripheral for acquiring the microphone signal. The `pdm_start` and `pdm_rec` utilities are used for clock generation and data acquisition. The PCM peripheral also performs the required decimation, providing 16-bit samples at a rate equal to one-sixteenth of the PDM clock frequency. [2]

6.1 Clock Generation and Data Acquisition

The PDM clock is generated on GPIO18 using the `pdm_start` utility. The microphone DATA output is connected to GPIO20, from which `pdm_rec` acquires the samples.

A clock of 2.4 MHz was requested. Due to the available clock divider in the Raspberry Pi, the actual clock generated was 2.347826 MHz. This lies within the standard operating range of the microphone. The corresponding output sample rate was calculated as

$$f_s = \frac{f_{\text{clock}}}{16} \tag{5}$$

giving

$$f_s = \frac{2.347826 \times 10^6}{16} \approx 146.739 \text{ kHz}. \tag{6}$$

The recorded output is provided as signed 16-bit samples in little-endian format. [3]

6.2 Signal Analysis

A Python-based tool, `pdm_tool.py`, was developed to analyse the samples obtained from `pdm_rec`. The tool includes four modes: `check`, `sweep`, `analyze`, and `wav`. For this method, the `check` and `analyze` modes were used.

The `check` mode records a quiet signal and a known 1 kHz tone and evaluates the captured samples. It calculates basic signal parameters such as mean, standard deviation, RMS level, and DC content. It also performs frequency-domain analysis using Welch's power spectral density method to determine the dominant frequency and its prominence above the noise floor. Waveform, spectrum, histogram, and spectrogram plots are generated for visual inspection.

The `analyze` mode allows an already recorded `.dat` file to be analysed without performing a new recording. The tool also uses the detected tone frequency to estimate the actual clock frequency of the system. [3]

6.3 Experimental Results

A 1 kHz tone was applied to the microphone and recorded using the PCM peripheral. The analysis result for one of the recordings is shown in Figure 1.

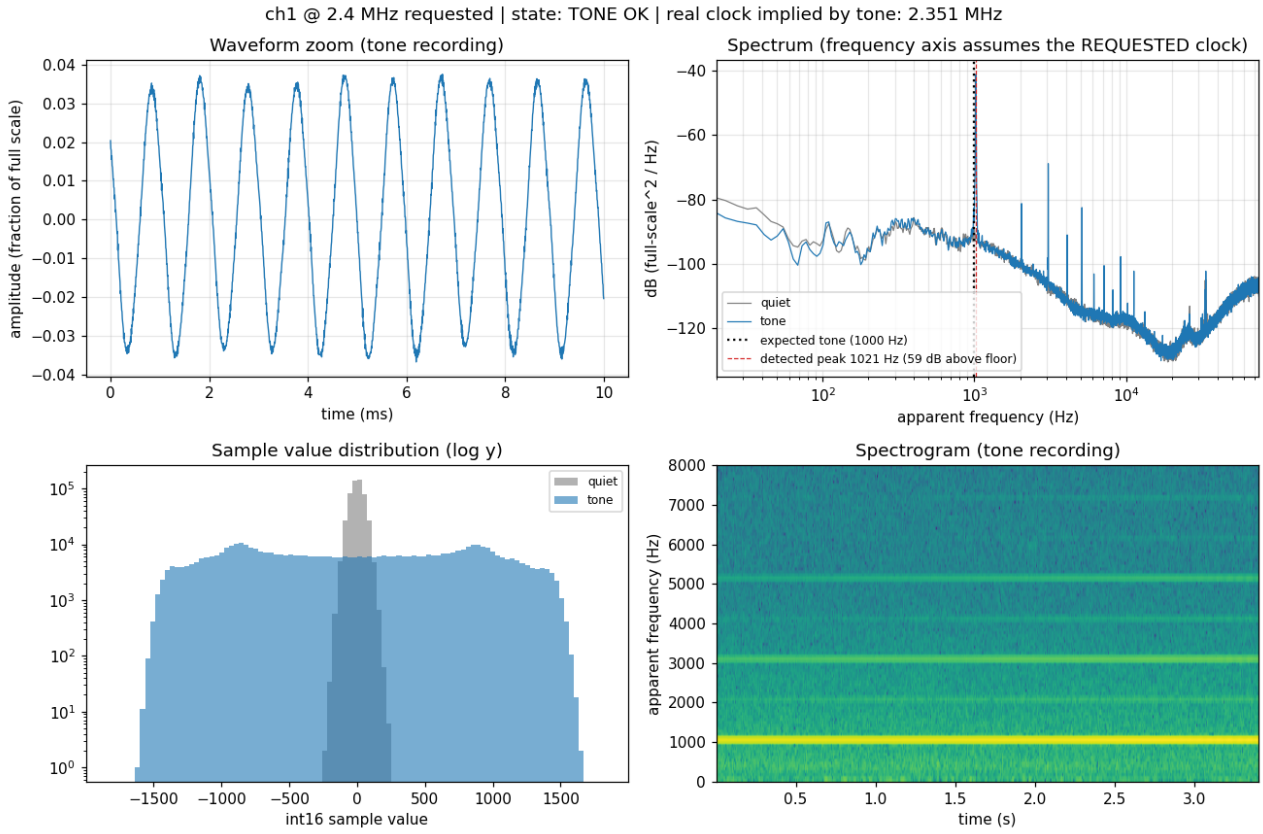


Figure 1: Analysis of the microphone output using the PCM peripheral.

The waveform shows a clear periodic signal corresponding to the applied tone. The spectrum shows a distinct peak at approximately 1021 Hz, with a prominence of about 59 dB above the surrounding noise floor. The histogram shows a wider distribution of sample values when the tone is

present compared to the quiet recording. The spectrogram also shows a continuous frequency component around 1 kHz throughout the recording.

Across repeated measurements, the detected peak remained approximately 1021 Hz, while the tone prominence varied between 58 and 63 dB. The tone-implied clock was approximately 2.351 MHz, which was close to the actual clock of 2.347826 MHz.

Table 3: Results of PCM Peripheral Based Acquisition

Parameter	**Measured Value**
Requested clock	2.4 MHz
Actual clock	2.347826 MHz
Output sample rate	146.739 kHz
Test tone	1 kHz
Detected peak	$\approx$1021 Hz
Tone prominence	58–63 dB
Output format	16-bit signed samples

A 4-second voice recording was also successfully captured and converted to a WAV file for playback. The audio could be played back, confirming that the acquired samples contained usable audio information.

References

[1] Knowles, “SPH0641LU4H-1 Digital MEMS Microphone Datasheet.”

[2] assi-dangomushi, “pdm_rec,” GitHub repository.

[3] PDM Tool, “pdm_tool.py,” Project implementation.